MOBILE COMPUTING

ASSIGNMENT 5 – CO5

ANDROID APPLICATION FEATURES & PERFORMANCE ENGINEERING

Submitted by: **B Chaithanya Prakash Reddy**

Register Number: **192425255**

Department: **AI&ML**

Course: **Mobile Computing**

INTRODUCTION

Modern mobile applications must remain responsive while working with limited battery, memory, network bandwidth and device resources. They must also protect private information and support devices with different hardware capabilities. Assignment 5 focuses on Android background work, safe synchronisation, reusable design patterns, computational tractability, approximate UI optimisation and multimedia engineering.

The report follows the assignment workflow: understand the problem, design a solution, check constraints, implement, measure, challenge the design and interpret the result. Part A presents architectural reasoning and language-independent pseudocode. Part B presents implementation plans, instrumentation, test workloads and result interpretation. Actual device measurements and screenshots must be inserted after running the experiments; they are not invented in this document.

OBJECTIVES

- Design Android Intent, Service and WorkManager flows that respect battery, connectivity and privacy constraints.
- Improve synchronisation by selecting safe records incrementally and handling conflicts.
- Identify a common framework for sensor collection, offline synchronisation and push notification delivery.
- Classify mobile workloads according to verification difficulty, resource needs and processing location.
- Design a practical heuristic for smooth scrolling under frame-time and battery limits.
- Implement and instrument background scheduling, adversarial workloads and exact-versus-approximate optimisation.
- Select video encoding and upload settings using measurable device and network constraints.

PART A – DESIGN, ANALYSIS & ARCHITECTURAL REASONING

TASK A1 – CONSTRAINT-AWARE INTENT AND SERVICE ARCHITECTURE

A1.1 – Problem Model

Input: sensor events, current network state, battery level, user preferences, permission state, Doze state and pending upload records.

Output: securely stored sensor records, successfully uploaded records and user notifications for critical events.

Decision at each stage: whether to collect a reading, store it locally, enqueue an upload, defer the upload, send a notification or cancel work.

Constraints: the required sensor permission must be granted; sensitive records must be encrypted; battery and network conditions must satisfy the selected work policy; background execution must follow Android restrictions; the application must not expose private data after logout.

Goal condition: all valid records are safely stored and eventually uploaded when permitted, critical events are delivered promptly through an allowed notification path, and no sensitive data is transmitted or exposed without authorisation.

A1.2 – Runtime State

- Work identifier and work type: sensor collection, upload or critical notification.

- Record identifier, encrypted payload reference, sensitivity level and creation timestamp.
- Work status: pending, selected, running, deferred, completed, failed or cancelled.
- Network type, connectivity status and measured quality at the decision point.
- Battery percentage, charging state and whether the device is in Doze mode.
- Permission and authentication state, including whether the user has logged out.
- Retry count, next eligible execution time and last error reason.
- Notification status and upload acknowledgement, so completed work is not repeated.

The state must be persistent for work that can survive process termination. A worker should reload the saved state rather than relying on variables that exist only in memory.

A1.3 – Feasibility and Pruning Conditions

A task is eligible only when its required permission is granted, the user session is valid, the payload is encrypted and the task's declared network and battery constraints are satisfied. A non-urgent upload may be deferred during Doze or poor connectivity. A task must be cancelled if the user logs out, revokes consent or invalidates the session. A critical notification may use an approved urgent path, but it must not bypass authentication or privacy rules.

A1.4 – Complete Architectural Pseudocode

```
ALGORITHM PROCESS_BACKGROUND_QUEUE(State)

INPUT:
    PendingWork, Battery, Network, Doze, Permissions, UserSession
OUTPUT:
    Completed, Deferred, Cancelled work and notification status

BEGIN
    WHILE PendingWork is not empty DO
        Work <- SELECT_NEXT_ELIGIBLE_OR_HIGHEST_PRIORITY(PendingWork)

        IF Work does not exist THEN
            WAIT for a relevant state change
            CONTINUE
        END IF

        IF UserSession is INVALID OR Work is no longer authorised THEN
            MARK Work as CANCELLED
            REMOVE sensitive temporary data
            RECORD cancellation reason
            CONTINUE
        END IF

        IF required permission is NOT GRANTED THEN
            MARK Work as DEFERRED or CANCELLED
            NOTIFY user when an explanation is appropriate
            CONTINUE
        END IF

        IF Work contains sensitive data AND payload is NOT ENCRYPTED THEN
            REJECT Work
            RECORD security failure
            CONTINUE
        END IF
```

```
        IF Work.Type = SENSOR_COLLECTION THEN
            Candidate <- Explicit Intent to collection component
            IF sensor permission and battery policy are feasible THEN
                START permitted collection operation
                STORE reading securely
                MARK Work as COMPLETED
            ELSE
                DEFER Work until eligible
            END IF

        ELSE IF Work.Type = DATA_UPLOAD THEN
            Candidate <- WorkManager worker with network constraint
            IF Network is suitable AND battery policy permits AND Doze policy allows
THEN
                ENQUEUE or RUN worker
                SEND encrypted record
                IF server acknowledgement received THEN
                    MARK record as UPLOADED
                    COMMIT sync state
                ELSE
                    RETAIN record and schedule bounded retry
                END IF
            ELSE
                DEFER upload and preserve encrypted record
            END IF

        ELSE IF Work.Type = CRITICAL_ALERT THEN
            Candidate <- permitted notification Intent
            IF notification permission and channel are available THEN
                POST critical notification
                MARK alert as DELIVERED
            ELSE
                RECORD undelivered alert and apply approved fallback
            END IF
        END IF

        IF operation fails after partial state change THEN
            RESTORE previous sync state
            RELEASE resources
            CANCEL incomplete operation
        END IF
    END WHILE

    RETURN final work status
END
```

Intent selection: use an explicit Intent when addressing a known internal component; use an implicit Intent only when a system or external handler should resolve the action; use an ordered broadcast only when ordered receiver processing is genuinely required. Use WorkManager for deferrable, constraint-based work that must survive process restarts. A foreground service is reserved for user-visible work that must continue immediately and is permitted by current Android rules.

A1.5 – Manual Execution Trace

1. Sensor reading arrives. The system checks permission, session validity and battery policy. The reading is encrypted and saved locally.
2. Connectivity changes from unavailable to Wi-Fi. The upload item becomes eligible and is selected by the scheduler.

3. The device enters Doze before the upload begins. The non-urgent upload is deferred; its encrypted record and retry state remain saved.
4. A critical sensor event occurs. The alert path checks notification permission and posts the critical notification through the permitted notification mechanism.
5. The user logs out. Pending uploads tied to the session are cancelled, temporary plaintext is cleared and the scheduler refuses further work for that session.

The trace demonstrates Choose → Check Constraint → Execute or Defer → Notify → Cancel / Backtrack. The upload is not discarded merely because the network is unavailable; it remains safely queued until a valid execution window exists.

TASK A2 – SAFE INCREMENTAL DATA SYNCHRONISATION

A2.1 – Problems in the Naïve Algorithm

- It attempts every record without first checking priority, size, network quality or battery.
- It may transmit sensitive data without verifying encryption and user authorisation.
- It does not handle duplicate uploads or server acknowledgements.
- It ignores record conflicts and may overwrite a newer server version.
- Continuing after a network failure wastes processing and may trigger repeated radio use.
- It has no clear distinction between a record that should be uploaded, deferred, discarded or resolved.

A2.2 – Incremental Constraint-Aware Synchronisation

```
ALGORITHM SAFE_INCREMENTAL_SYNC(LocalStore, RemoteAPI, State)

BEGIN
    SyncSet <- empty
    FOR each record R in priority order DO
        IF R is invalid or expired under the retention policy THEN
            DISCARD R with audit reason
            CONTINUE
        END IF

        IF R is sensitive AND R is not encrypted THEN
            REJECT R and raise security event
            CONTINUE
        END IF

        IF user session is invalid THEN
            DEFER or CANCEL session-bound record
            CONTINUE
        END IF

        IF server version conflicts with local version THEN
            RESOLVE using version, timestamp and domain rule
            IF conflict cannot be safely resolved THEN
                DEFER R for review
                CONTINUE
            END IF
        END IF

        IF network quality is insufficient OR battery is below policy threshold THEN
            DEFER R and preserve local copy
            CONTINUE
        END IF
```

```
            ADD R to SyncSet
            UPLOAD R using stable record ID
            IF acknowledgement received THEN
                MARK R as synced
            ELSE
                RETAIN R as pending and stop current batch if link is unstable
            END IF
        END FOR
        RETURN SyncSet and updated local state
    END
```

A2.3 – Example of Pruning

Consider five records: R1 is a high-priority encrypted critical alert of 2 KB; R2 is a routine encrypted record of 8 KB; R3 is a sensitive unencrypted record of 4 KB; R4 is a low-priority image record of 5 MB; R5 is an encrypted medium-priority record of 10 KB. Under weak network and low battery, R1 may use an approved urgent path, R2 and R5 can be deferred, R3 is rejected until encrypted, and R4 is deferred because its size is unsuitable for the current link.

The branch for R3 can terminate immediately: transmitting an unencrypted sensitive record violates the security constraint regardless of available bandwidth or battery. Further exploration of upload timing cannot make that branch safe; encryption or an authorised alternative must happen first.

A2.4 – Changed Requirement: Offline-First Eventual Consistency

The original pruning rule must be revised if it discards records merely because they are not currently uploadable. Offline-first operation requires preserving valid local changes and synchronising them later. The violated assumption is that a record unable to upload now has no future value. Under eventual consistency, temporary network failure is a reason to defer, not discard. Conflict resolution and stable record identifiers become essential.

TASK A3 – COMMON MOBILE BACKGROUND-WORK PATTERN

A3.1 – Common Pattern

Sensor collection, offline data synchronisation and push-notification delivery appear different, but all follow a common state-driven background-work pattern. Each has a trigger, a stored state, a candidate operation, feasibility checks, an execution step, a completion condition and a cancellation or recovery path.

Sensor collection

State includes permission, sampling interval, last reading and battery status. The decision is whether to sample now or wait. Candidate choices are a scheduled worker or an appropriate foreground service. Feasibility checks include sensor permission, user consent and battery policy. The goal is a valid encrypted reading. Cancellation stops sampling and releases sensor resources.

Data synchronisation

State includes pending records, server versions, network quality and retry count. The decision is which record to upload. Candidate choices are upload, defer or resolve conflict. Feasibility checks include connectivity, battery, encryption and valid session. The goal is server acknowledgement. Cancellation retains the local record and restores its pending status.

Push notification

State includes alert priority, notification permission, channel and delivery status. The decision is whether to post an alert or use a permitted fallback. Feasibility checks include permission, valid content and alert policy.

The goal is successful user notification. Cancellation removes stale pending alerts and prevents duplicate delivery.

A3.2 – Generic Background-Work Framework

```
ALGORITHM SCHEDULE_OR_EXECUTE(State)

BEGIN
    EVENT <- SELECT_NEXT_EVENT(State)

    IF EVENT does not exist THEN
        WAIT for trigger
        RETURN
    END IF

    ACTIONS <- GENERATE_CANDIDATE_ACTIONS(EVENT)

    FOR each ACTION in ACTIONS DO
        IF NOT AUTHORISED(ACTION, State) THEN
            PRUNE ACTION
            CONTINUE
        END IF

        IF NOT RESOURCE_FEASIBLE(ACTION, State) THEN
            DEFER ACTION with next eligible condition
            CONTINUE
        END IF

        SAVE checkpoint of current state
        RESULT <- EXECUTE ACTION

        IF RESULT is successful THEN
            COMMIT new state
            RECORD completion
            RETURN SUCCESS
        ELSE
            RESTORE checkpoint
            RELEASE resources
            RECORD failure and retry policy
        END IF
    END FOR

    RETURN DEFERRED_OR_FAILED
END
```

A3.3 – Role of Undo and Cancellation

Undo and cancellation prevent partial operations from leaving inconsistent state. During data synchronisation, a request may fail after a local record has been marked as sent but before the server acknowledges it. If the application does not restore the pending status, the record may be lost. A safe design commits the synced state only after acknowledgement and uses a stable identifier so retries do not create duplicates.

TASK A4 – TRACTABILITY: LOCAL, CLOUD AND HYBRID PROCESSING

A mobile performance problem must be defined by measurable requirements, not simply by saying that an application runs slowly. Useful measures include latency, throughput, energy per task, memory, bandwidth, privacy risk, reliability and the number of users or devices supported.

1. On-device image classification using a tiny model

Solution: run a compact quantised model locally and return a class label with confidence. It can be efficiently verified using a fixed test set and accuracy metrics. Resource class: On-device feasible, with battery considerations. Local execution reduces network dependency and keeps images on the device.

2. Cloud-based speech recognition

Solution: capture audio, securely transmit it to a speech-recognition service and return recognised text. Verification uses labelled audio and word-error rate. Resource class: Network-bound. Cloud offloading can use larger models, but latency, bandwidth, service availability and privacy must be considered.

3. Peer-to-peer file sharing among nearby devices

Solution: discover nearby peers, authenticate them and transfer files using a secure local connection. Verification checks file hashes, transfer completion and peer authentication. Resource class: Hybrid required. Local transfer reduces internet dependence, but discovery, interference and device compatibility can be difficult.

4. End-to-end encrypted messaging

Solution: encrypt messages on the sender device and decrypt only on the recipient device using authenticated key exchange and message encryption. Verification can test protocol properties, known test vectors and security assumptions. Resource class: Security-critical. The cloud may relay ciphertext but must not receive plaintext keys.

5. Real-time multiplayer game-state synchronisation

Solution: send compact state updates, timestamps and player inputs through a low-latency server or peer network, with interpolation and prediction. Verification uses latency, jitter, packet loss and state divergence. Resource class: Network-bound and battery-critical. A hybrid design may combine authoritative cloud state with local prediction.

6. Large-scale offline map rendering with vector tiles

Solution: store selected vector tiles locally, decode and render them on-device, and fetch missing tiles when connectivity returns. Verification uses visual correctness, tile coverage, frame time and memory use. Resource class: Hybrid required. Local rendering supports offline use, while cloud delivery supplies map updates and larger data.

Why Local vs Cloud Matters

Local processing reduces network delay and can protect raw data, but it consumes device CPU, memory and battery. Cloud processing can scale compute resources and centralise updates, but introduces network dependence, service cost, latency and data-transfer risk. Hybrid processing places time-sensitive or privacy-sensitive work locally and offloads suitable heavy work when connectivity, consent and security conditions allow.

TASK A5 – UI PERFORMANCE AND APPROXIMATION

A5.1 – Exact Perspective

An exact search could enumerate every combination of decoding resolution, cache size and prefetch distance, measure a cost function and select the minimum-cost configuration. If there are q quality levels, c cache sizes and p prefetch distances, the search evaluates q × c × p configurations for each device profile. As the number of feed items and device profiles increases, testing every combination becomes expensive in time, energy and memory. Runtime conditions also change while the feed is scrolling.

A5.2 – Greedy / Adaptive Strategy

```
ALGORITHM ADAPTIVE_FEED_RENDERER(State)

INPUT:
    FrameTime, Battery, MemoryPressure, Network, FeedPosition
OUTPUT:
    DecodeQuality, CacheSize, PrefetchWindow

BEGIN
    Initialise quality, cache and prefetch settings
    REPEAT
        Measure FrameTime, Battery and MemoryPressure

        IF FrameTime exceeds 16 ms OR MemoryPressure is HIGH THEN
            Reduce decode quality by one level
            Evict least-recently-used cached items
            Reduce prefetch window
        ELSE IF Battery is LOW THEN
            Reduce video resolution and background prefetch
            Batch upcoming requests
        ELSE IF FrameTime is comfortably below target
                AND MemoryPressure is LOW THEN
            Increase prefetch window by one small step
            Increase quality only if battery permits
        END IF

        SAVE selected parameters
        UPDATE state using latest measurements
    UNTIL stable for several measurement windows
          OR user leaves feed
    RETURN selected parameters
END
```

A5.3 – What Is Sacrificed?

A feasible smooth-scrolling result means the renderer meets a practical frame-time target under current conditions. It does not prove that the selected parameters minimise total energy for every possible combination of quality, cache size and prefetch distance. A heuristic gains speed by examining a small number of adjustments rather than all combinations, but it may settle at a local or merely acceptable solution. The sacrifice is a global optimality guarantee.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

Part B requires source code or a detailed simulation, test inputs, expected behaviour, actual outputs, instrumentation and interpretation. The following design provides a practical implementation plan. Replace

every [MEASURE] or [INSERT] field with evidence collected from Android Studio, an emulator or a real device.

TASK B1 – IMPLEMENT AND INSTRUMENT BACKGROUND ARCHITECTURE

B1.1 – Implementation Design

Create a WorkManager-based Android prototype. Each work item contains a unique ID, type, priority, encrypted payload reference and constraints. A worker checks network state, battery policy, permission and session validity before executing. Record counters for enqueued, deferred, cancelled and successful items, along with maximum concurrent workers and final backlog.

```
ALGORITHM INSTRUMENTED_WORKER(WorkItem, DeviceState)

BEGIN
    INCREMENT enqueued counter when item is created
    RECORD battery, network, Doze and permission state

    IF permission revoked OR session invalid THEN
        INCREMENT cancelled counter
        RETURN CANCELLED
    END IF

    IF network or battery constraint is not satisfied THEN
        INCREMENT deferred counter
        RETURN RETRY_LATER
    END IF

    INCREMENT active worker count
    UPDATE maximum concurrent worker count

    RESULT <- execute upload or collection task

    DECREMENT active worker count
    IF RESULT is acknowledged THEN
        INCREMENT successful counter
        RETURN SUCCESS
    ELSE
        INCREMENT deferred counter
        RETURN RETRY_LATER
    END IF
END
```

B1.2 – Validation Scenarios

6. Good connectivity and battery: enqueue several encrypted records on Wi-Fi with sufficient battery. Expected behaviour is successful execution and a reduced pending backlog.
7. Doze or low battery: schedule non-urgent work while the device is idle or below the selected battery threshold. Expected behaviour is deferral, not data loss.
8. Permission revoked mid-execution: revoke the required permission during a test. Expected behaviour is safe cancellation, resource release and no further sensor access.

These tests verify normal execution, constraint handling and safe response to a boundary condition. Record the exact device state, work counters and log output for each run.

B1.3 – FIFO and Priority Scheduling

Version 1 processes work items in arrival order (FIFO). Version 2 assigns a score based on criticality, age and estimated cost. One possible rule is PriorityScore = 5 × Criticality + 2 × AgeLevel − SizePenalty. The score is only a selection heuristic; constraints are still checked before execution. A critical alert can be selected earlier, while a large low-priority upload may wait for Wi-Fi.

B1.4 – Measurement and Interpretation

For the same workload, record enqueued items, deferred/cancelled items, successful executions and peak concurrent workers for both versions. Also record time to deliver the first critical item and final residual backlog.

FIFO results: [MEASURE]. Priority results: [MEASURE]. Interpretation: [EXPLAIN WHETHER THE PRIORITY RULE IMPROVED CRITICAL DELIVERY AND WHETHER IT INCREASED WAIT TIME FOR LOW-PRIORITY ITEMS]. Do not claim improvement unless the measured results support it.

TASK B2 – ADVERSARIAL WORKLOAD

Construct two workloads with the same number of records and identical constraint thresholds. Workload A should contain mostly small encrypted records, stable Wi-Fi and adequate battery. Workload B should contain large records, weak or intermittent connectivity, low battery, sensitive records awaiting encryption and a mixture of priorities.

Prediction

Workload A is expected to complete with few deferrals because most items satisfy the constraints. Workload B is expected to require more deferrals, retries and conflict checks because the structure of the records and environment creates more constraint failures. Equal input size does not imply equal work.

Execution and Comparison

Record the number of work items processed, deferrals, successful deliveries, retries, bytes uploaded and time spent with the radio active. Workload A results: [MEASURE]. Workload B results: [MEASURE].

The difficult workload has more constraint-ineligible records and unstable network windows. The scheduler repeatedly evaluates state and preserves deferred records. This shows why input count alone does not describe computational or energy effort: record size, priority, sensitivity, connectivity and conflict structure affect the amount of work.

TASK B3 – EXACT VS APPROXIMATE UI / GRAPHICS OPTIMISATION

B3.1 – Exact and Approximate Implementations

For a small feed, define quality levels Q = {low, medium, high}, cache sizes C = {small, medium, large} and prefetch distances P = {1, 2, 3}. The exact method enumerates all 27 combinations and selects the configuration with the lowest cost. A sample cost function is Cost = 4 × DroppedFrames + 2 × EnergyUnits + MemoryPenalty.

The approximate method begins at medium quality and adjusts one parameter at a time using frame time, battery and memory pressure. It stops when performance is stable for several windows or the user leaves the feed.

B3.2 – Three Small Test Instances

9. Instance 1: low-complexity images, high battery and low memory pressure. Expected: exact search may choose high quality and a larger cache; the heuristic may reach a similar setting.

10. Instance 2: mixed images and videos, medium battery and moderate memory pressure. Expected: exact search may find a lower-cost balance; the heuristic may stop at medium quality.
11. Instance 3: large video items, low battery and high memory pressure. Expected: exact search may choose low quality, small cache and short prefetch; the heuristic should reduce settings as pressure rises.

For each instance record Exact cost: [MEASURE], Approximate cost: [MEASURE], Match: [YES/NO]. Use the same cost function and input assumptions for both methods.

B3.3 – Counter-example

A greedy strategy may reduce quality immediately when frame time briefly exceeds 16 ms, even though the true best configuration is to keep high quality and reduce cache size. If cache eviction alone would remove memory pressure with a smaller quality penalty, the first greedy choice moves away from the global optimum. Trace each adjustment and compare the final cost with exhaustive search.

B3.4 – Increase Problem Size and Interpret

Increase the number of feed items and quality levels. Record configurations evaluated by exact search, number of heuristic quality changes, cache evictions and prefetch adjustments. Exact evaluation grows with the product of the number of choices for each parameter. The heuristic evaluates only the current state and a small number of adjustments.

Exact operation count: [MEASURE]. Approximate operation count: [MEASURE]. Exact cost: [MEASURE]. Approximate cost: [MEASURE]. The approximate method gains computational efficiency and faster adaptation, but sacrifices a guarantee of global minimum cost. It is acceptable for continuously changing feeds where a good feasible setting is more valuable than spending time proving an optimum.

TASK B4 – MULTIMEDIA FRAMEWORK SELECTION

B4.1 – Practical Encoding Strategy

For video capture and upload, choose resolution, bitrate and upload timing based on device capability, battery and network quality. A practical rule is to start with a medium profile, reduce resolution and bitrate on weak networks or low battery, and upload in a batch when connectivity is stable. Preserve an emergency or user-selected high-quality mode when required.

```
ALGORITHM SELECT_VIDEO_PROFILE(Device, Network, Battery)

BEGIN
    IF Battery is LOW OR Device temperature is HIGH THEN
        Resolution <- 480p
        Bitrate <- LOW
        UploadTiming <- DEFER_UNTIL_STABLE_NETWORK
    ELSE IF Network is WEAK THEN
        Resolution <- 480p
        Bitrate <- LOW
        UploadTiming <- UPLOAD_SMALL_CHUNKS
    ELSE IF Network is MODERATE THEN
        Resolution <- 720p
        Bitrate <- MEDIUM
        UploadTiming <- BATCH_UPLOAD
    ELSE
        Resolution <- 1080p
        Bitrate <- HIGH
        UploadTiming <- UPLOAD_WHEN_READY
    END IF
```

```
    Estimate encoding time and energy
    Estimate battery impact
    RETURN Resolution, Bitrate, UploadTiming
END
```

The numeric profiles are illustrative starting points. The final values must be calibrated for the selected device and application quality requirement.

B4.2 – Challenging Input

A device may have a high-end camera and high battery but a very weak uplink. A heuristic that selects 1080p based mainly on device capability may produce a large file that takes too long to upload and consumes substantial radio energy. A better rule includes measured uplink throughput, expected clip size and upload deadline, not only hardware capability.

B4.3 – Engineering Decision Note

- Exhaustive search is unattractive at production scale because combinations of resolution, frame rate, codec, bitrate, chunk size and upload timing multiply across thousands of different device and network profiles.
- The heuristic provides a fast feasible selection under current conditions, but it does not guarantee minimum energy or the best possible visual quality.
- Its advantage is bounded decision work and quick adaptation to battery, temperature and network changes.
- Exact or cloud-assisted optimisation may be appropriate offline, for a small number of profiles, or when enough time and stable connectivity are available to evaluate many configurations.
- Before deployment, monitor encoding time, upload completion time, retry rate, battery consumption, thermal throttling, quality metrics, crash rate and user-visible delays.

ADDITIONAL ENGINEERING EXPLANATION AND TEST GUIDANCE

1. Reproducible Experiment Method

- Use the same device or emulator profile, operating-system version, application build, input data and test duration when comparing two approaches. Record the device model, available memory, battery level, network type and thermal state.
- Run each experiment multiple times. Background updates, runtime compilation, thermal throttling and radio variation can change measurements. Report the observations and a mean or median rather than relying on one run.
- Separate predictions from measured results. Write the expected outcome before the experiment, then compare it with the actual logs and profiler evidence. Explain any difference rather than altering the result to fit the prediction.
- Keep an experiment record containing purpose, configuration, input, steps, expected result, actual result, evidence filename and interpretation.

2. Understanding Performance Metrics

- Frame time measures how long the application takes to produce a frame. A 16 ms target is a useful approximation for a 60 Hz display, but device refresh rates and rendering pipelines may differ.
- Throughput measures the amount of useful data processed per unit time. Latency measures how long one operation takes. A system can have high throughput and still have poor latency for an urgent record.
- Peak concurrent workers indicates how many tasks are active together. More workers do not always improve performance; they can compete for CPU, memory, storage and network resources.

- Battery percentage is not a precise energy measurement. Wake-up count, radio-active time, CPU use and measured energy estimates provide more useful evidence about why battery use changed.

3. Security and Data Integrity

- Encrypt sensitive records before placing them in persistent local storage. Keep encryption keys in platform-protected key storage where available.
- Use a stable unique record ID for every synchronised item. The server should treat repeated uploads with the same ID as the same logical operation.
- Mark a record as synchronised only after receiving a valid server acknowledgement. If the acknowledgement is lost, retry with the same ID rather than creating a new record.
- After logout or permission revocation, stop session-dependent work, clear temporary plaintext and prevent old workers from uploading data under an invalid session.

4. Engineering Trade-offs

A design decision usually improves one property while affecting another. Increasing video quality can improve visual experience but increase encoding time, file size and upload energy. Increasing prefetch distance can reduce waiting during scrolling but increase memory and network use. Increasing background worker concurrency can improve throughput but may increase contention and battery consumption. The correct setting depends on the application requirement and must be justified by measured evidence.

PART B – EVIDENCE CHECKLIST

- Attach source code or a detailed executable simulation for background work, synchronisation and UI optimisation.
- Include test inputs and identify the expected behaviour before showing actual output.
- Capture work counters, deferrals, cancellations, successful executions and peak concurrent workers.
- Show FIFO and priority scheduling results using the same workload.
- Include two same-size adversarial workloads and explain the structural difference.
- Report exact and approximate costs for at least three small instances and one counter-example.
- Record operation counts as problem size increases; do not report only wall-clock time.
- For multimedia selection, show chosen resolution, bitrate, upload timing and estimated device impact.
- Replace every [MEASURE] and [INSERT] field with genuine experiment evidence before submission.

OVERALL CONCLUSION

Assignment 5 demonstrates that efficient mobile engineering requires more than making an application run. Background work must be selected using explicit state and feasibility conditions. Synchronisation must preserve data safely, resolve conflicts and defer work when conditions are unsuitable. Sensor collection, data sync and notifications share a common state-driven scheduling pattern. Local, cloud and hybrid processing must be selected according to latency, privacy, battery and scalability requirements.

For UI rendering and multimedia, exact optimisation can become too expensive as the number of parameters and device profiles grows. Adaptive heuristics can provide fast feasible choices, but they do not guarantee a global optimum. The engineer must measure frame time, energy, memory, network behaviour and quality, then explain the trade-offs and limitations of the chosen design.

REFERENCES

12. SIMATS Engineering, ITA03-MP Assignment 5 – CO5: Android Application Features & Performance Engineering, supplied course handout.

13. Android Developers Documentation: Intents and Intent Filters; Services; WorkManager; Background Tasks; Doze and App Standby; Notifications; Permissions.
14. Android Developers Documentation: App Architecture, Data Storage, Security Best Practices and Performance Profiling.
15. Android Developers Documentation: Android Studio Profiler, Memory Profiler, CPU Profiler and Network Inspector.
16. Android Developers Documentation: MediaCodec, CameraX and background execution guidance.